Side-Channel Attacks and Countermeasures

Tyler Vander Mooren

CYB 412 - Secure Hardware

Professor Mirza Mohd Shahriar Maswood

April 28, 2026

Abstract

Side-channel attacks show that cryptographic security can fail even when the algorithm itself is mathematically strong. The weakness is not always in AES, RSA or the size of the key space. In secure hardware, the weakness can come from the physical behavior of the device running the algorithm. Power consumption, electromagnetic emissions, timing differences and faulty outputs can all reveal information that should remain hidden. This paper examines how side-channel attacks take advantage of that gap between ideal cryptography and real hardware implementation. Power analysis, electromagnetic analysis, timing attacks and fault injection attacks are compared by looking at the type of physical behavior each attack uses and how that behavior can expose secret data or bypass security. The main finding is that side-channel resistance cannot depend on a single countermeasure. Masking, hiding, shielding, constant-time execution, redundancy, fault detection and validation testing all reduce risk in different ways. Strong hardware security requires layered defenses matched to the device's real attack surface. Overall, side-channel attacks demonstrate that secure hardware must protect not only the cryptographic algorithm, but also the physical execution of that algorithm.

Side-Channel Attacks and Countermeasures

Cryptography is often treated as a mathematical security problem. If an encryption algorithm has a large enough key space and no practical mathematical weakness, it is commonly viewed as secure. In real hardware, however, this view is incomplete. A cryptographic device does not only perform abstract mathematical operations. It also consumes electrical power, emits electromagnetic signals, takes measurable time to complete operations and reacts to environmental changes such as voltage, clock or temperature variation. These physical behaviors can reveal information about the secret data being processed inside the device. A side-channel attack takes advantage of this gap between the ideal algorithm and the real implementation. Instead of attacking the cryptographic algorithm directly, the attacker studies the physical leakage created while the device operates. This makes side-channel attacks especially important in secure hardware, embedded systems, smart cards, IoT devices, FPGAs and cryptographic modules. These systems often store or process sensitive keys in environments where an attacker may eventually gain physical access to the device.

This issue matters because modern systems increasingly depend on hardware as a security foundation. Secure boot, encrypted storage, authentication tokens, payment cards, mobile devices, medical sensors, industrial systems and IoT platforms all depend on cryptographic operations being protected in practice. If an attacker can recover a secret key by measuring power traces, observing electromagnetic emissions or manipulating a device into producing faulty outputs, then the cryptographic algorithm no longer protects the system by itself. Implementation attacks can defeat otherwise strong cryptographic systems because the attacker targets how the system behaves in the real world, not only the mathematical design (Paar & Pelzl, 2010). Side-channel attacks expose a weakness that traditional cryptographic analysis

does not fully address. Strong algorithms are still vulnerable when the hardware running them leaks useful physical information. For that reason, the focus is on how power analysis, electromagnetic analysis, timing attacks and fault injection attacks work in practice. Defending against these attacks requires more than one countermeasure because resistance has to be built through layered design choices, including secure implementation, physical protection, fault detection and validation testing.

Methodology

Side-channel attacks are best analyzed by separating the cryptographic algorithm from the hardware that runs it. The point is not to assume that the algorithm itself is mathematically broken. The real question is what happens when that algorithm is placed inside a physical device that consumes power, produces electromagnetic signals, takes measurable time to run and can be disrupted by changes in voltage, clock or environmental conditions. That approach fits the difference between normal cryptanalysis and implementation attacks, where the weakness comes from the way the system behaves in practice rather than the design of the algorithm itself (Paar & Pelzl, 2010). The analysis compares the main side-channel categories by looking at the physical behavior each attack uses. Power analysis depends on changes in electrical consumption. Electromagnetic analysis depends on emissions from the device or board. Timing attacks depend on differences in execution time. Fault injection depends on forcing the hardware into an abnormal state and then using the faulty behavior to learn something useful. These attacks are grouped this way because they represent different paths to the same goal, which is extracting secret information or bypassing security without directly defeating the cryptographic algorithm (Bhunia & Tehranipoor, 2019). Countermeasures are evaluated by asking whether they reduce the leakage, hide the leakage, detect manipulation or make the device fail safely. Masking,

hiding, balanced design, shielding, constant-time execution, redundancy and fault detection are not treated as separate magic fixes. They are compared as layers that reduce risk in different ways. That matters because a defense against one side channel may not protect against another. A device protected against timing leakage may still leak through power. A device protected against passive measurement may still be vulnerable to fault injection. For that reason, the strongest approach is not choosing one defense but selecting several defenses for the attack surface of the hardware being protected.

Results

Side-channel attacks work because physical devices reveal more than the final input and output of a cryptographic operation. In theory, an attacker should only see ciphertext, plaintext or public values. In practice, the attacker may also see how much power the device uses, how long the operation takes, what electromagnetic signals it emits or how it behaves when the hardware is disturbed. That extra information is what becomes the side channel. The danger is not that the cryptographic algorithm suddenly becomes weak. The danger is that the implementation gives the attacker another way around the algorithm. Power analysis is one of the clearest examples of this problem. A device does not consume power randomly. Its power use changes as transistors switch during computations. When cryptographic operations depend on secret values, those changes can sometimes reveal patterns related to the key. In simple power analysis, the attacker looks directly at power traces and tries to connect visible changes in power consumption to operations inside the device. If an implementation performs different steps depending on key bits, the power trace may reveal those differences. More advanced power attacks do not need the leakage to be obvious in one trace. An attacker can collect many measurements and use statistical comparison to find small patterns that repeat across executions. This is what makes

power analysis especially serious. A single trace may look harmless, but repeated measurements can expose information that was not visible at first. The device may appear to be running a secure algorithm, but the physical power behavior can still reveal part of the secret state. Power side channels are tied to the way hardware activity changes during computation, which is why cryptographic hardware must be designed with leakage in mind (Bhunia & Tehranipoor, 2019).

Electromagnetic analysis follows a similar idea, but it uses emissions instead of direct power measurement. As the current moves through the circuits, the device can produce electromagnetic signals. Those signals can sometimes be measured near the chip, the board or even a specific area of the device. This can make electromagnetic attacks more targeted than basic power analysis. Instead of measuring the power behavior of the entire system, an attacker may focus on a smaller region where the cryptographic operation is taking place. The issue with electromagnetic leakage is that it turns physical layout into part of the security problem. A design may be mathematically correct and still leak because traces, components or circuit regions produce measurable emissions. That means defense cannot only happen in the code or algorithm. Shielding, layout choices, noise reduction, balanced design and physical separation of sensitive operations do all matter. In other words, electromagnetic attacks show that secure hardware is not just about what the device computes, but also where and how that computation happens.

Timing attacks use a different kind of leakage. Instead of measuring power or emissions, the attacker measures how long an operation takes. If the amount of time changes based on secret data, the timing behavior becomes useful. This can happen when a cryptographic operation uses branches, memory accesses or arithmetic operations that depend on secret values. Even small timing differences can matter if the attacker can collect enough observations. Timing attacks are important because they may not always require direct physical probing. Some timing differences

can be observed through normal device interaction and in some cases may even be visible remotely. That makes timing leakage different from attacks that require opening the device or attaching probes. The defense is to make execution behavior independent of secret data. Constant time execution, fixed operation paths and avoiding secret-based branching all reduce timing leakage. The cost is that the implementation may do extra work even when a faster path exists. That tradeoff is usually worth it because the faster path may also reveal the secret.

Fault injection attacks are also different because the attacker is no longer just watching the device. The attacker is actively trying to make it misbehave. This can be done by manipulating voltage, clock signals, temperature, electromagnetic conditions or other physical inputs. The goal is to cause a useful error. A useful error could mean skipping an instruction, corrupting a calculation, bypassing a check or producing a faulty cryptographic output. This matters because many security mechanisms assume that the device executes correctly. A secure boot process assumes the signature verification step actually happened. A cryptographic module assumes the calculation was completed correctly. A password check assumes the comparison was not skipped. Fault injection attacks challenge those assumptions because the attacker can force the hardware into the wrong state at the right time, the device may produce information or behavior that should never be available during normal operation. OWASP treats timing and glitching attacks as physical implementation risks against processing units, including attacks that can retrieve key material or manipulate cryptographic calculations (OWASP Foundation, 2021).

The countermeasures for these attacks fall into several practical categories. Some defenses try to reduce the leakage; this is done by balanced circuit design, shielding and careful layout to make useful physical signals harder to observe. While some try to hide leakage by masking, randomization and noise generation to try to break the relationship between the secret

value and the measured signal. Other defenses focus on manipulation. Fault sensors, duplicate calculations and error checks try to detect when the device has been disturbed. A strong design also needs safe failure behavior because detecting a fault does not help much if the device continues running and releases a faulty output. None of these defenses should be treated as a single fix. A device protected against timing leakage can still leak through power. A device protected against simple power analysis may still be vulnerable to more advanced statistical analysis. A device that resists passive observation may still be vulnerable to fault injection. This is why side-channel resistance must be layered. The defense must match the physical attack surface of the device and it must be tested. NIST SP 800-140F connects non-invasive attack mitigation to cryptographic module validation, which shows that side-channel resistance belongs in real evaluation and testing processes, not just theory (Schaffer, 2020).

That point is especially important for embedded systems, IoT devices and sensor networks. These devices are often small, resource constrained and physically exposed after deployment. In sensor networks, this creates a serious security concern because devices may operate outside controlled environments, making physical access by an attacker more realistic. In those environments, side-channel attacks are not just lab demonstrations. They are practical threats against devices that may be hard to patch, replace or physically protect once deployed (Monjur et al., 2022).

Discussion

The main issue with side-channel attacks is that they force a different way of thinking about security. In normal cryptographic analysis, the question is usually whether the algorithm can be broken through mathematical weakness, brute force or poor key management. Side-channel attacks shift that question. The algorithm may be strong, the key space may be large and

the design may look secure on paper, but the hardware can still leak information while it operates. That means the implementation becomes part of the attack surface. This is why side-channel attacks are so important in secure hardware. The attacker does not need to defeat AES, RSA or another cryptographic algorithm directly. Instead, the attacker looks for physical behavior that should not reveal anything useful but does. Power consumption, electromagnetic emissions, timing differences and faulty outputs become extra sources of information. These side channels are not supposed to be part of the security model, but in real devices they cannot be ignored.

The risk also depends heavily on the environment where the device is used. A server locked in a controlled data center faces a different physical threat model than a smart card, sensor node, IoT device or embedded controller deployed in the field. Many hardware devices are small, inexpensive and difficult to physically protect after deployment. In sensor networks especially, devices may operate outside controlled environments, making physical access by an attacker more realistic. Once an attacker can handle the device, probe it, power it, reset it or disturb it, side-channel attacks become much more practical. The countermeasure problem is that each defense only covers part of the attack surface. Constant-time execution can reduce timing leakage, but it does not automatically stop power or electromagnetic leakage. Shielding may reduce electromagnetic emissions, but it does not fix secret-dependent branching or poor fault handling. Masking can make power analysis harder, but it does not make a device immune to all forms of statistical attack. Fault detection can identify abnormal behavior, but only if the device responds safely when a fault is detected. A countermeasure that detects a fault but still releases corrupted output is not enough.

This is why side-channel resistance must be layered. A secure design should reduce leakage where possible, hide leakage when it cannot be fully removed, detect manipulation and fail safely when something abnormal happens. These defenses work best when they reinforce each other. For example, a cryptographic device may use constant-time code to reduce timing leakage, masking to reduce key-dependent power patterns, shielding and layout changes to reduce electromagnetic leakage and redundancy to detect fault injection. None of those defenses are perfect by itself, but together they make the attack more expensive, less reliable and harder to repeat. There is also a practical tradeoff, a stronger side-channel protection usually costs something. Masking and balanced design can increase circuit complexity. Shielding and layout changes can increase cost and design effort. Redundancy can increase area, power consumption or execution time. Constant-time execution can reduce performance because the device may need to perform extra operations even when a faster path exists. These tradeoffs matter because many devices most exposed to side-channel attacks are also the most constrained. IoT devices, smart cards and sensor nodes often have limited power, limited memory, limited processing capability and strict cost targets. That tradeoff does not make side-channel protection optional. It means the defense must match the realistic threat model. A low-cost consumer sensor may not need the same protection level as a payment card or hardware security module, but it still needs some level of side-channel awareness if it stores keys or performs authentication. A device that protects financial transactions, secure boot keys or medical data needs stronger protection because the impact of key recovery or bypass is higher. Hardware security is not about adding every possible defense everywhere. It is about understanding what the device protects, how exposed it is and what attack methods are realistic.

Another important point is that side-channel security cannot be trusted only because the design claims to include countermeasures, it must be tested. A design may include masking, shielding or fault detection and still leak in unexpected ways. Small implementation details can create a measurable pattern. A compiler optimization can reintroduce timing leakage. A board layout decision can create stronger electromagnetic emissions. A fault sensor may miss a narrow glitch. This is why validation matters. NIST's non-invasive attack mitigation guidance shows that resistance to these types of attacks belongs in cryptographic module testing, not just in theory or design documentation (Schaffer, 2020).

Overall, side-channel attacks show that secure hardware must be judged by how it behaves in the real world. Strong algorithms are necessary, but they are not enough by themselves. The device must also protect the physical execution of those algorithms. The strongest approach is not to search for one perfect countermeasure, because that countermeasure does not exist. The better approach is layered design, realistic testing and an honest threat model that treats physical leakage as part of the security problem. In that sense, side-channel security is not separate from hardware security; it is one of the clearest examples of why hardware security matters in the first place.

References

Bhunia, S., & Tehranipoor, M. H. (2019). *Hardware security: A hands-on learning approach*. Morgan Kaufmann.

Monjur, M. M. R., Heacock, J., Calzadillas, J., Mahmud, M. S., Roth, J., Mankodiya, K., Sazonov, E., & Yu, Q. (2022). Hardware Security in Sensor and its Networks. *Frontiers in Sensors*, *3*. https://doi.org/10.3389/fsens.2022.850056

OWASP Foundation. (2021). *Processing units*. *OWASP IoT Security Testing Guide*. https://owasp.org/owasp-istg/03_test_cases/processing_units/index.html

Paar, C., & Pelzl, J. (2014). *Understanding cryptography: A Textbook for Students and Practitioners*. Springer.

Schaffer, K. (2021). *CMVP Approved Non-Invasive Attack Mitigation Test Metrics: CMVP Validation Authority Updates to ISO/IEC 24759*. https://doi.org/10.6028/NIST.SP.800-140Fr1-draft